

```

# -*- coding: utf-8 -*-
"""
KOREYA-GA so'z boti
=====
Bu fayl Telegram botning "miyasi". U
words.json fayldan barcha
koreyscha so'zlarni o'qiydi va
foydalanuvchiga 3 xil rejimda ko'rsatadi:
    1) Mavzular bo'yicha ko'rish (Boblar)
    2) Flashcard (kartochka) o'yini
    3) Qidiruv

Eslatma: Token (botning "parol"i)
xavfsizlik uchun shu faylga emas,
balki muhit o'zgaruvchisiga (environment
variable) yoziladi.
Pastdagi "BOT_TOKEN" qismini ko'ring.
"""

import json
import logging
import os
import random

from telegram import InlineKeyboardButton,
InlineKeyboardMarkup, Update, WebAppInfo
from telegram.constants import ParseMode
from telegram.ext import (
    Application,
    CallbackQueryHandler,

```

```

        CommandHandler,
        ContextTypes,
        MessageHandler,
        filters,
    )

# -----
# -----
# 1) SOZLAMALAR
# -----
# -----

# Token serverga "environment variable"
sifatida qo'yiladi (xavfsizroq).
# Agar shu yerda sinab ko'rmoqchi
bo'lsangiz, pastdagi qatorni
# BOT_TOKEN = "0000000000:AAAA....." kabi
to'g'ridan-to'g'ri yozib turishingiz
mumkin,
# lekin keyin serverga yuklashda uni
o'chirib, muhit o'zgaruvchisiga o'tkazing.
BOT_TOKEN = os.environ.get("BOT_TOKEN",
"BU_YERGA_TOKENINGIZNI_QOYING")

# Saytingiz manzili (Mini App). GitHub
Pages'dan olingan https:// havola.
WEBAPP_URL = os.environ.get("WEBAPP_URL",
"https://xatbosim9-
ai.github.io/Koreya_bot/")

logging.basicConfig(
    format="%(asctime)s - %(name)s - %(
levelname)s - %(message)s",
    level=logging.INFO,

```

```

)
logger = logging.getLogger(__name__)

# -----
# -----
# 2) MA'LUMOTLARNI YUKLASH (words.json)
# -----
# -----

DATA_PATH =
os.path.join(os.path.dirname(os.path.abspath(
__file__)), "words.json")
with open(DATA_PATH, encoding="utf-8") as
f:
    CHAPTERS = json.load(f)

# Barcha so'zlarni bitta uzun ro'yxatga ham
yig'ib qo'yamiz (Flashcard va Qidiruv
uchun)
ALL_WORDS = []
for ci, ch in enumerate(CHAPTERS):
    for wi, w in enumerate(ch["words"]):
        ALL_WORDS.append({**w,
"chapter_idx": ci, "word_idx": wi,
"chapter_title": ch["titleUz"]})

POS_EMOJI = {
    "noun": "🟦 Ot",
    "verb": "🟥 Fe'l",
    "adj": "🟪 Sifat",
    "adv": "🟡 Ravish",
    "idiom": "🟠 Ibora",
}

```

```

# -----
# -----
# 3) YORDAMCHI FUNKSIYALAR (matn va
# tugmalarni tayyorlash)
# -----
# -----

def main_menu_kb():
    """Bosh menyu tugmalari."""
    keyboard = [
        [InlineKeyboardButton("🌐 Saytni
ochish",
web_app=WebAppInfo(url=WEBAPP_URL))],
        [InlineKeyboardButton("📖
Mavzular", callback_data="menu:chapters")],
        [InlineKeyboardButton("📄
Flashcard", callback_data="flash:start")],
        [InlineKeyboardButton("🔍
Qidirish", callback_data="menu:search")],
    ]
    return InlineKeyboardMarkup(keyboard)

def chapters_list_kb():
    """Barcha 12 ta mavzu ro'yxati tugma
sifatida."""
    keyboard = []
    for i, ch in enumerate(CHAPTERS):
        label = f"{i+1:02d}. {ch['title']}
({len(ch['words'])} so'z)"

    keyboard.append([InlineKeyboardButton(label
, callback_data=f"chapter:{i}:0")])

```

```

keyboard.append([InlineKeyboardButton("🏠
Bosh menyu", callback_data="menu:main")])
return InlineKeyboardMarkup(keyboard)

```

```

def word_card_text(ch_idx, w_idx):
    """Bitta so'z kartochkasining matnini
    tayyorlaydi."""
    ch = CHAPTERS[ch_idx]
    w = ch["words"][w_idx]
    pos = POS_EMOJI.get(w["posClass"],
w["pos"])
    text = (
        f"*{ch['title']}* -
{w_idx+1}/{len(ch['words'])}\n\n"
        f"🇰🇷 *{w['kr']}* ({pos})\n"
        f"🇺🇿 {w['uz']}\n\n"
        f"_Misol:_\n"
        f"{w['exKr']}\n"
        f"{w['exUz']}"
    )
    return text

```

```

def word_card_kb(ch_idx, w_idx):
    """So'z kartochkasi ostidagi
    navigatsiya tugmalari (Oldingi/Keyingi)."""
    n = len(CHAPTERS[ch_idx]["words"])
    prev_i = (w_idx - 1) % n
    next_i = (w_idx + 1) % n
    keyboard = [
        [
            InlineKeyboardButton("⬅️

```

```
Oldingi", callback_data=f"chapter:{ch_idx}:{prev_i}"),
        InlineKeyboardButton("Keyingi
➡", callback_data=f"chapter:{ch_idx}:{next_i}"),
    ],
    [InlineKeyboardButton("📖 Mavzular ro'yxati", callback_data="menu:chapters")],
    [InlineKeyboardButton("🏠 Bosh menyu", callback_data="menu:main")],
]
return InlineKeyboardMarkup(keyboard)
```

```
def flash_front_kb():
    keyboard = [
        [InlineKeyboardButton("↺ Javobni ko'rsatish", callback_data="flash:flip")],
        [InlineKeyboardButton("🔀 Aralashtirish", callback_data="flash:shuffle")],
        [InlineKeyboardButton("🏠 Bosh menyu", callback_data="menu:main")],
    ]
    return InlineKeyboardMarkup(keyboard)
```

```
def flash_back_kb():
    keyboard = [
        [InlineKeyboardButton("➡ Keyingi karta", callback_data="flash:next")],
        [InlineKeyboardButton("🔀 Aralashtirish", callback_data="flash:shuffle")],
    ]
```

```

        [InlineKeyboardButton("🏠 Bosh
menu", callback_data="menu:main")],
    ]
    return InlineKeyboardMarkup(keyboard)

async def safe_edit(query, text, kb):
    """Xabarni tahrirlaydi; Markdown xato
bersa, oddiy matn bilan urinib ko'radi."""
    try:
        await query.edit_message_text(text,
reply_markup=kb,
parse_mode=ParseMode.MARKDOWN)
    except Exception:
        await query.edit_message_text(text,
reply_markup=kb)

# -----
# -----
# 4) BUYRUQLAR (/start)
# -----
# -----

async def start(update: Update, context:
ContextTypes.DEFAULT_TYPE):
    context.user_data["awaiting_search"] =
False
    text = (
        "안녕하세요! 🙌\n\n"
        "**KOREYA-가 so'z bot*'iga xush
kelibsiz!\n"
        f"Bu yerda {len(CHAPTERS)} ta mavzu
va {len(ALL_WORDS)} ta so'z bor.\n\n"

```

```

        "Admin: @whsdvv\n\n"
        "Quyidagilardan birini tanlang:"
    )
    await update.message.reply_text(text,
reply_markup=main_menu_kb(),
parse_mode=ParseMode.MARKDOWN)

# -----
-----
# 5) TUGMA BOSILGANDA ISHLAYDIGAN FUNKSIYA
# -----
-----

async def button_handler(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    query = update.callback_query
    await query.answer() # Telegramga
    "bosildi" deb signal beramiz
    data = query.data

    # --- Bosh menyu ---
    if data == "menu:main":

context.user_data["awaiting_search"] =
False
        text = "Quyidagilardan birini
tanlang:"
        await safe_edit(query, text,
main_menu_kb())
        return

    # --- Mavzular ro'yxati ---
    if data == "menu:chapters":

```

```

        await safe_edit(query, "📖 Mavzuni
tanlang:", chapters_list_kb())
        return

    # --- Bitta mavzudagi bitta so'z
    kartochkasi ---
    if data.startswith("chapter:"):
        _, ch_idx, w_idx = data.split(":")
        ch_idx, w_idx = int(ch_idx),
int(w_idx)
        await safe_edit(query,
word_card_text(ch_idx, w_idx),
word_card_kb(ch_idx, w_idx))
        return

    # --- Flashcard rejimini boshlash ---
    if data == "flash:start":
        order = list(range(len(ALL_WORDS)))
        random.shuffle(order)
        context.user_data["flash_order"] =
order
        context.user_data["flash_pos"] = 0
        await show_flash_front(query,
context)
        return

    if data == "flash:next":
        order =
context.user_data.get("flash_order")
        if not order:
            order =
list(range(len(ALL_WORDS)))
            random.shuffle(order)

```

```

context.user_data["flash_order"] = order
        context.user_data["flash_pos"]
= 0
        else:
            context.user_data["flash_pos"]
= (context.user_data.get("flash_pos", 0) +
1) % len(order)
            await show_flash_front(query,
context)
            return

        if data == "flash:shuffle":
            order = list(range(len(ALL_WORDS)))
            random.shuffle(order)
            context.user_data["flash_order"] =
order
            context.user_data["flash_pos"] = 0
            await show_flash_front(query,
context)
            return

        if data == "flash:flip":
            await show_flash_back(query,
context)
            return

        # --- Qidiruv rejimiga o'tish ---
        if data == "menu:search":

context.user_data["awaiting_search"] = True
            text = "🔍 Qidirmoqchi bo'lgan
so'zni (koreyscha yoki o'zbekcha) yozib
yuboring:"
            await safe_edit(query, text,

```

```

InlineKeyboardMarkup(
    [[InlineKeyboardButton("🏠 Bosh
menu", callback_data="menu:main")]]
))
return

```

```

async def show_flash_front(query, context):
    order =
context.user_data["flash_order"]
    pos = context.user_data["flash_pos"]
    w = ALL_WORDS[order[pos]]
    p = POS_EMOJI.get(w["posClass"],
w["pos"])
    text = (
        f"📌 Flashcard
({pos+1}/{len(order)})\n\n"
        f"🇰🇷 *{w['kr']}* ({p})\n\n"
        f"_Javobni ko'rish uchun pastdagi
tugmani bosing_ 📌"
    )
    await safe_edit(query, text,
flash_front_kb())

```

```

async def show_flash_back(query, context):
    order =
context.user_data["flash_order"]
    pos = context.user_data["flash_pos"]
    w = ALL_WORDS[order[pos]]
    p = POS_EMOJI.get(w["posClass"],
w["pos"])
    text = (
        f"📌 Flashcard

```

```

({pos+1}/{len(order)})\n\n"
    f"🇰🇷 *{w['kr']}* ({p})\n"
    f"🇺🇿 {w['uz']}\n\n"

f"_Misol:_{w['exKr']}\n{w['exUz']}"
)
await safe_edit(query, text,
flash_back_kb())

# -----
-----
# 6) ODDIY MATN XABAR KELGANDA (Qidiruv
uchun)
# -----
-----

async def text_handler(update: Update,
context: ContextTypes.DEFAULT_TYPE):
    if not
context.user_data.get("awaiting_search"):
        await update.message.reply_text(
            "Iltimos, tugmalardan
foydalaning 📁 yoki /start bosing.",
            reply_markup=main_menu_kb(),
        )
    return

    query_text =
update.message.text.strip().lower()
    results = [
        w for w in ALL_WORDS
        if query_text in w["kr"].lower()
        or query_text in w["uz"].lower()

```

```

        or query_text in w["exUz"].lower()
    ]

    if not results:
        await update.message.reply_text(
            f"❌ \"{update.message.text}\"
            bo'yicha hech narsa topilmadi. Boshqa so'z
            bilan urinib ko'ring.",
        )
        return

    results = results[:10] # bittada ko'p
    bo'lib ketmasligi uchun
    for w in results:
        p = POS_EMOJI.get(w["posClass"],
            w["pos"])
        text = (
            f"*{w['chapter_title']}*\n\n"
            f"🇰🇷 *{w['kr']}* ({p})\n"
            f"🇺🇿 {w['uz']}\n\n"

            f"_Misol:_{w['exKr']}\n{w['exUz']}"
        )
        try:
            await
            update.message.reply_text(text,
                parse_mode=ParseMode.MARKDOWN)
        except Exception:
            await
            update.message.reply_text(text)

    context.user_data["awaiting_search"] =
    False
    await update.message.reply_text("Yana

```

```

qidirish uchun 🔍 tugmasini bosing:",
reply_markup=main_menu_kb())

# -----
# -----
# 7) BOTNI ISHGA TUSHIRISH
# -----
# -----

def main():
    if BOT_TOKEN ==
"BU_YERGA_TOKENINGIZNI_QOYING":
        raise SystemExit(
            "❌ BOT_TOKEN topilmadi!
README.md faylidagi 1-bosqichni bajaring."
        )

    app =
Application.builder().token(BOT_TOKEN).build()

    app.add_handler(CommandHandler("start",
start))

    app.add_handler(CallbackQueryHandler(button
_handler))

    app.add_handler(MessageHandler(filters.TEXT
& ~filters.COMMAND, text_handler))

    logger.info("Bot ishga tushdi...")
    app.run_polling()

```

```
if __name__ == "__main__":  
    main()
```